Studio Nx

S1-07 샘플 라이브러리 UI — 함수 설명 합본 (JS 193~210)

2026-10-17

목차

1. ai-notes/javascript/S1-07 - 샘플 라이브러리 UI/193_api_fetchSampleList.md
2. ai-notes/javascript/S1-07 - 샘플 라이브러리 UI/194_api_uploadSample.md
3. ai-notes/javascript/S1-07 - 샘플 라이브러리 UI/195_api_fetchSampleFileUrl.md
4. ai-notes/javascript/S1-07 - 샘플 라이브러리 UI/196_api_fetchSampleWaveform.md
5. ai-notes/javascript/S1-07 - 샘플 라이브러리 UI/197_api_renameSample.md
6. ai-notes/javascript/S1-07 - 샘플 라이브러리 UI/198_api_setSampleTags.md
7. ai-notes/javascript/S1-07 - 샘플 라이브러리 UI/199_api_deleteSample.md
8. ai-notes/javascript/S1-07 - 샘플 라이브러리 UI/200_api_fetchTags.md
9. ai-notes/javascript/S1-07 - 샘플 라이브러리 UI/201_sequencer-ui_initSamplePanel.md
10. ai-notes/javascript/S1-07 - 샘플 라이브러리 UI/202_sequencer-ui_renderSampleList.md
11. ai-notes/javascript/S1-07 - 샘플 라이브러리 UI/203_sequencer-ui_drawSampleWaveform.md
12. ai-notes/javascript/S1-07 - 샘플 라이브러리 UI/204_sequencer-ui_applySampleFilters.md
13. ai-notes/javascript/S1-07 - 샘플 라이브러리 UI/205_sequencer-ui_toggleSamplePreview.md
14. ai-notes/javascript/S1-07 - 샘플 라이브러리 UI/206_sequencer-ui_uploadQueue.md
15. ai-notes/javascript/S1-07 - 샘플 라이브러리 UI/207_sequencer-ui_renderUploadRow.md
16. ai-notes/javascript/S1-07 - 샘플 라이브러리 UI/208_sequencer-ui_updateStorageBar.md
17. ai-notes/javascript/S1-07 - 샘플 라이브러리 UI/209_sequencer-ui_toggleEmptyState.md
18. ai-notes/javascript/S1-07 - 샘플 라이브러리 UI/210_sequencer-ui_startSampleEdit.md

JS 193 — 샘플 목록 불러오기 (`fetchSampleList`)

항목	내용
파일	`studionx/frontend/static/js/core/api.js`
작업	S1-07 샘플 라이브러리: 첫 번째 함수 (FR-224 목록·검색)
작성	Claude (AI), 2026-10-17, 사용자 검토
브랜치	`feature/phase1/s1-07-sample-library-ui`
다음	194 `uploadSample`

1. 하는 일

샘플 목록을 서버에서 받아 옵니다. `GET /samples` 에 검색어·태그 번호·쪽 번호·쪽 크기를 붙여 부르고, 돌려받은 JSON(`items`·`total`·`page`·`size`)을 그대로 돌려줍니다.

```
fetchSampleList({ size: 50 })
  → GET /api/v1/samples?size=50
  → { items: [SampleResponse, …], total: 128, page: 1, size: 50 }
```

2. 코드

```
export async function fetchSampleList(params = {}) {
  const qs = new URLSearchParams();
  if (params.query) qs.set('query', params.query);
  if (params.tag != null) qs.set('tag', String(params.tag));
  if (params.page != null) qs.set('page', String(params.page));
  if (params.size != null) qs.set('size', String(params.size));
  const tail = qs.toString();
  return api.get('/samples' + (tail ? '?' + tail : ''));
}
```

3. 왜 이렇게

선택	이유
`URLSearchParams` 로 쿼리 조립	값이 비어 있으면 아예 안 붙습니다. 손으로 `?`·`&` 를 잇지 않아 실수가 없습니다
값이 있을 때만 `set`	`query=''`·`tag=0` 같은 빈 조건을 서버로 보내지 않습니다. `tag != null` 은 0번 태그도 살립니다
`size` 를 넘겨받음	화면(201)이 50(최대)로 부릅니다. 기본 10을 그대로 두면 목록이 10줄만 보입니다
`api.get` 재사용	토큰 재발급·401 재시도·오류 변환이 공통 함수에 이미 있습니다

4. 시험

확인	결과
`fetchSampleList({ size: 50 })` → `/samples?size=50` (스텁 검증)	✓
목록 3건을 받아 `#file-list` 에 3줄이 그려짐 (202 연동)	✓
실패 시 예시 줄을 지우지 않음 (401 은 조용히, 그 외 `showToast`)	✓

JS 194 — 샘플 업로드 (`uploadSample`, `xhrError`)

항목	내용
파일	`studionx/frontend/static/js/core/api.js`
작업	S1-07 샘플 라이브러리: 두 번째 함수 (FR-221·226 업로드)
작성	Claude (AI), 2026-10-17, 사용자 검토
브랜치	`feature/phase1/s1-07-sample-library-ui`
다음	195 `fetchSampleFileUrl`

1. 하는 일

고른 파일 하나를 `POST /samples`(multipart, `file`)로 올리고, 진행률을 콜백으로 알려 줍니다. 취소(AbortSignal)를 받으면 그 업로드만 멈추고, 끝나면 올린 샘플 한 줄(`SampleResponse`)을 돌려줍니다.

```
uploadSample(file, { signal, onProgress(loaded, total) })
  → POST /api/v1/samples  (multipart/form-data)
  → SampleResponse
```

2. 코드

```
export function uploadSample(file, options = {}) {
  const signal = options.signal;
  const onProgress = options.onProgress;
  return new Promise((resolve, reject) => {
    if (signal && signal.aborted) { reject(new ApiError(0, 'ABORTED', 'Upload cancelled.')); return; }

    function send(retried) {
      const form = new FormData();
      form.append('file', file, file.name);
      const xhr = new XMLHttpRequest();
      xhr.open('POST', `${API_BASE}/samples`);
      xhr.withCredentials = true;
      if (accessToken) xhr.setRequestHeader('Authorization', `Bearer ${accessToken}`);
      xhr.setRequestHeader('Accept', 'application/json');
      if (xhr.upload && onProgress) {
        xhr.upload.addEventListener('progress', (e) => {
          if (e.lengthComputable) onProgress(e.loaded, e.total);
        });
      }
      xhr.addEventListener('load', () => {
        if (xhr.status === 401 && !retried) {
          refreshAccessToken().then((ok) => (ok ? send(true) : reject(xhrError(xhr))));
          return;
        }
        if (xhr.status >= 200 && xhr.status < 300) resolve(parse(xhr));
        else reject(xhrError(xhr));
      });
      xhr.addEventListener('error', () => reject(new ApiError(0, 'NETWORK_ERROR', 'Network error. Please
```

```
try again.')));
      xhr.addEventListener('abort', () => reject(new ApiError(0, 'ABORTED', 'Upload cancelled.')));
      if (signal) signal.addEventListener('abort', () => xhr.abort());
      xhr.send(form);
    }

    if (!accessToken) refreshAccessToken().then(() => send(false));
    else send(false);
  });
}

function xhrError(xhr) {
  let data = null;
  try { data = xhr.responseText ? JSON.parse(xhr.responseText) : null; } catch { data = null; }
  const e = data && data.error;
  if (e && e.code) return new ApiError(xhr.status, e.code, e.message == null ? '' : e.message, e.details
== null ? null : e.details);
  return new ApiError(xhr.status, 'INTERNAL_ERROR', 'Something went wrong. Please try again.');
}
```

3. 왜 이렇게

선택	이유
`XMLHttpRequest` (fetch 아님)	`fetch` 는 업로드 **진행률**(FR-226)을 제공하지 않습니다. `xhr.upload` 의 `progress` 이벤트가 필요합니다
`FormData` 로 `file` 한 칸	백엔드 `POST /samples` 가 `multipart/form-data` 의 `file` 필드를 받습니다(177·192 문서)
`AbortSignal` → `xhr.abort()`	업로드 줄의 취소(×) 버튼과 연결됩니다. 취소는 실패가 아니라 `code:'ABORTED'` 로 구분합니다
401 이면 재발급 후 1회 재시도	Access Token 이 메모리에만 있어 만료될 수 있습니다. `apiFetch` 와 같은 규칙입니다
오류를 `ApiError` 로 통일	fetch 용 `toApiError` 와 같은 모양(`status`·`code`·`message`)이라 화면(206)이 한 곳에서 분기합니다

4. 시험

확인	결과
파일 올리면 `#upload-list` 줄이 생기고 0% → 100% 로 채워짐 (스텁 XHR)	✓
끝나면 줄이 사라지고 새 샘플이 목록 맨 위에 붙음 (`New Loop`)	✓
취소(×) 시 줄이 사라지고 목록이 그대로임	✓ (설계)

JS 195 — 샘플 미리듣기 주소 (`fetchSampleFileUrl`)

항목	내용
파일	`studionx/frontend/static/js/core/api.js`
작업	S1-07 샘플 라이브러리: 두 번째 함수 (FR-224 미리듣기)
작성	Claude (AI), 2026-10-17, 사용자 검토
브랜치	`feature/phase1/s1-07-sample-library-ui`
다음	196 `fetchSampleWaveform`

1. 하는 일

샘플의 오디오 파일(`GET /samples/{id}/file`)을 토큰을 붙여 받아 `Blob` 으로 만들고, `objectURL` 로 바꿔 돌려줍니다. 이 주소를 `<audio src>` 에 넣어 미리듣기(205)에 씁니다. 같은 샘플은 한 번만 받습니다.

2. 코드

```
const sampleFileUrls = new Map();

export async function fetchSampleFileUrl(sampleId) {
  const key = String(sampleId);
  const hit = sampleFileUrls.get(key);
  if (hit) return hit;

  if (!accessToken) await refreshAccessToken();

  const send = () => {
    const headers = { Accept: '*/*' };
    if (accessToken) headers.Authorization = `Bearer ${accessToken}`;
    return fetch(`${API_BASE}/samples/${encodeURIComponent(key)}/file`, {
      headers: headers,
      credentials: 'same-origin',
    });
  };

  let res = await send();
  if (res.status === 401 && (await refreshAccessToken())) res = await send();
  if (!res.ok) throw await toApiError(res);

  const url = URL.createObjectURL(await res.blob());
  sampleFileUrls.set(key, url);
  return url;
}
```

3. 왜 이렇게

선택	이유
blob → objectURL	Access Token 이 메모리에만 있어 `<audio src="/samples/1/file">` 처럼 바로 걸면 토큰이 안 실립니다. 그래서 토큰을 붙여 받은 뒤 주소로 바꿉니다
`Map` 으로 캐시	같은 줄을 여러 번 재생할 때 다시 받지 않습니다. 창을 닫을 때까지 유지됩니다
401 이면 재발급 후 1 회 재시도	파일 응답은 `apiFetch` 를 거치지 않아 재발급 규칙을 여기서 따로 겁니다
`Accept: */*`	JSON 이 아니라 오디오 바이너리를 받으므로 JSON 을 요구하지 않습니다

4. 시험

확인	결과
▶ 를 누르면 `/samples/{id}/file` 요청이 나감 (스텁 검증, 호출 1회)	✓
두 번째 재생은 캐시로 요청이 늘지 않음	✓ (설계)

JS 196 — 샘플 파형 데이터 (`fetchSampleWaveform`)

항목	내용
파일	`studionx/frontend/static/js/core/api.js`
작업	S1-07 샘플 라이브러리: 두 번째 함수 (FR-223 파형)
작성	Claude (AI), 2026-10-17, 사용자 검토
브랜치	`feature/phase1/s1-07-sample-library-ui`
다음	197 `renameSample`

1. 하는 일

샘플의 파형(peaks) 데이터를 받아 옵니다. 백엔드가 미리 계산해 둔 값을 JSON 으로 주면(74 문서 §FR-223), 화면은 그 값을 캔버스에 그립니다(203).

```
fetchSampleWaveform(1)
  → GET /api/v1/samples/1/waveform
  → { peaks: [[-0.31, 0.42], [-0.12, 0.55], …] }   // [최솟값, 최댓값] 쌍의 배열
```

2. 코드

```
export async function fetchSampleWaveform(sampleId) {
  return api.get('/samples/' + encodeURIComponent(sampleId) + '/waveform');
}
```

3. 왜 이렇게

선택	이유
값은 JSON(`peaks`)	오디오 디코딩·다운샘플을 브라우저에서 하지 않습니다. 가벼운 숫자 배열만 받아 그립니다
`[최솟값, 최댓값]` 쌍	한 칸(세로줄)에 위·아래를 함께 그려 파형 모양을 그대로 살립니다(203)
`api.get` 재사용	토큰·오류 처리를 공통 함수에 맡깁니다

4. 시험

확인	결과
목록 3줄이 각각 파형을 받아 캔버스 폭·높이가 0보다 큼	✓
파형이 없거나 실패하면 캔버스를 만들지 않고 세로 무늬 그대로 둠	✓ (설계)

JS 197 — 샘플 이름 바꾸기 (`renameSample`)

항목	내용
파일	`studionx/frontend/static/js/core/api.js`
작업	S1-07 샘플 라이브러리: 두 번째 함수 (FR-225 이름 변경)
작성	Claude (AI), 2026-10-17, 사용자 검토
브랜치	`feature/phase1/s1-07-sample-library-ui`
다음	198 `setSampleTags`

1. 하는 일

샘플 이름을 바꿉니다. `PATCH /samples/{id}` 에 `{ name }` 을 보내고, 바뀐 샘플 한 줄(붙어 있는 태그 포함)을 돌려받습니다.

```
renameSample(1, 'Kick 909 v2')
  → PATCH /api/v1/samples/1  { "name": "Kick 909 v2" }
  → SampleResponse
```

2. 코드

```
export async function renameSample(sampleId, name) {
  return api.patch('/samples/' + encodeURIComponent(sampleId), { name: name });
}
```

3. 왜 이렇게

선택	이유
`PATCH` (일부 수정)	이름만 바꾸므로 `PUT`(전체 교체)이 아닙니다. 서버도 이름만 받습니다(177 문서)
바뀐 한 줄을 돌려받음	편집 상자를 닫으며 그 줄만 다시 그리면(210) 됩니다. 목록 전체를 다시 받지 않습니다
이름 길이 검사는 서버가	최대 200자(`SAMPLE_NAME_MAX`) 검사는 백엔드가 합니다. 화면은 `maxLength=200` 로 미리 막습니다

4. 시험

확인	결과
이름을 'Kick 909 v2' 로 저장 → `PATCH /samples/1` 호출	✓
저장 뒤 첫 줄 이름이 'Kick 909 v2' 로 바뀜	✓

JS 198 — 샘플 태그 설정 (`setSampleTags`)

항목	내용
파일	`studionx/frontend/static/js/core/api.js`
작업	S1-07 샘플 라이브러리: 두 번째 함수 (FR-225 태그)
작성	Claude (AI), 2026-10-17, 사용자 검토
브랜치	`feature/phase1/s1-07-sample-library-ui`
다음	199 `deleteSample`

1. 하는 일

샘플에 붙은 태그를 통째로 바꿉니다. `PUT /samples/{id}/tags` 에 `{ tag_ids }` 를 보내고, 새로 붙은 태그가 담긴 샘플 한 줄을 돌려받습니다. 보낸 목록이 그 샘플의 태그 **전부**가 됩니다.

```
setSampleTags(1, [2, 5])
  → PUT /api/v1/samples/1/tags  { "tag_ids": [2, 5] }
  → SampleResponse
```

2. 코드

```
export async function setSampleTags(sampleId, tagIds) {
  return api.put('/samples/' + encodeURIComponent(sampleId) + '/tags', { tag_ids: tagIds });
}
```

3. 왜 이렇게

선택	이유
`PUT` (전체 교체)	태그는 붙이기·떼기가 섞여 순서 문제가 생깁니다. 화면이 최종 목록을 한 번에 보내면 서버가 그대로 맞춥니다
빈 목록이면 모두 뗌	`tag_ids: []` 가 "태그 없음"의 뜻입니다. 편집 상자에서 모두 끄면 이렇게 보냅니다
번호(`tag_id`) 배열	태그 이름은 바뀔 수 있어 화면은 번호로 다룹니다. 이름은 `/tags`(200) 목록에서 찾습니다

4. 시험

확인	결과
편집 상자에서 태그를 바꾸면 `PUT /samples/1/tags` 호출	✓
저장 뒤 그 줄에 새 태그가 반영됨	✓

JS 199 — 샘플 지우기 (`deleteSample`)

항목	내용
파일	`studionx/frontend/static/js/core/api.js`
작업	S1-07 샘플 라이브러리: 두 번째 함수 (FR-225 삭제)
작성	Claude (AI), 2026-10-17, 사용자 검토
브랜치	`feature/phase1/s1-07-sample-library-ui`
다음	200 `fetchTags`

1. 하는 일

샘플을 지웁니다(소프트 삭제). `DELETE /samples/{id}` 가 204(본문 없음)로 끝나면 성공입니다.

```
deleteSample(1)
  → DELETE /api/v1/samples/1
  → 204 (본문 없음 → null)
```

2. 코드

```
export async function deleteSample(sampleId) {
  return api.delete('/samples/' + encodeURIComponent(sampleId));
}
```

3. 왜 이렇게

선택	이유
되돌림은 화면이	서버는 소프트 삭제(잠시) 후 바로 지웁니다. 지우기 전 확인은 화면(209 문서의 205 계열)이 `confirm` 으로 합니다
204 는 null	`apiFetch` 가 204 를 null 로 돌려줍니다. 화면은 실패(throw)만 잡으면 됩니다
지운 뒤 목록 재정렬은 화면이	그 줄만 지우고 저장 용량을 빼면 됩니다(209). 목록을 다시 받지 않습니다

4. 시험

확인	결과
× 를 누르고 확인하면 `DELETE /samples/1` 호출	✓
줄이 사라지고(3 → 2) 저장 용량이 820 KB → 703 KB 로 줄어듦	✓

JS 200 — 태그 목록 불러오기 (`fetchTags`)

항목	내용
파일	`studionx/frontend/static/js/core/api.js`
작업	S1-07 샘플 라이브러리: 두 번째 함수 (FR-224 태그)
작성	Claude (AI), 2026-10-17, 사용자 검토
브랜치	`feature/phase1/s1-07-sample-library-ui`
다음	201 `initSamplePanel`

1. 하는 일

고를 수 있는 태그 목록을 받아 옵니다. `GET /tags` 가 번호·이름·분류를 돌려주면, 편집 상자(210)의 태그 버튼과 태그 필터(204)의 순환 목록에 씁니다.

```
fetchTags()
  → GET /api/v1/tags
  → { items: [{ tag_id: 1, name: 'Techno', category: 'GENRE' }, …] }
```

2. 코드

```
export async function fetchTags() {
  return api.get('/tags');
}
```

3. 왜 이렇게

선택	이유
분류(`category`)까지 받음	편집 상자에서 장르 태그를 다르게 보여 줄 수 있습니다(`GENRE` → `is-genre` 색)
실패해도 목록은 보여 줌	태그를 못 받아도 샘플은 봐야 합니다. 201 이 이 실패를 삼키고 빈 목록으로 둡니다

4. 시험

확인	결과
편집 상자에 태그 버튼 2개가 생기고 붙은 태그가 눌린 상태로 표시됨	✓
`+ Tag` 순환에 태그 이름이 순서대로 들어감	✓

JS 201 — 샘플 라이브러리 시작 (`initSamplePanel` · `loadSampleList`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-ui.js`
작업	S1-07 샘플 라이브러리: 아홉 번째 함수 (FR-221~227 패널 시작)
작성	Claude (AI), 2026-10-17, 사용자 검토
브랜치	`feature/phase1/s1-07-sample-library-ui`
다음	202 `renderSampleList`

1. 하는 일

왼쪽 샘플 패널을 시작합니다. 목업 예시 업로드 줄을 비우고, 패널의 모든 버튼을 연결(1165)한 뒤, 태그(200)와 목록(193)을 차례로 불러 그립니다. 둘 중 하나가 실패해도 패널은 그대로 씁니다.

```
initUI (sequencer-ui.js:15)
└ initSamplePanel
   ├ #upload-list 비우기 (목업 예시 줄 제거)
   ├ bindSamplePanelEvents()      ← 검색·필터·업로드·드롭·삭제·편집 연결
   ├ readStorageLimit()           ← 화면에 적힌 "… / 1 GB" 를 읽어 둠
   ├ fetchTags() → sampleState.tags
   └ loadSampleList() → fetchSampleList({ size: 50 }) → renderSampleList
```

2. 코드

```
async function initSamplePanel() {
  const listEl = document.getElementById('file-list');
  if (!listEl) return;
  const uploadEl = document.getElementById('upload-list');
  if (uploadEl) { uploadEl.textContent = ''; uploadEl.hidden = true; }   // 목업 예시 줄 비우기
  bindSamplePanelEvents();
  readStorageLimit();
  try {
    const tagRes = await fetchTags();
    sampleState.tags = tagRes && tagRes.items ? tagRes.items : [];
  } catch (e) {
    sampleState.tags = [];   // 태그가 없어도 목록은 보여 줍니다
  }
  await loadSampleList();
}

async function loadSampleList() {
  try {
    const res = await fetchSampleList({ size: 50 });
    renderSampleList(res && res.items ? res.items : [], res ? res.total : 0);
  } catch (e) {
    if (!e || e.status !== 401) showToast('Could not load your samples');
```

```
  }
}
```

3. 왜 이렇게

선택	이유
`initUI` 에서 부름	패널은 한 번만 시작하면 됩니다. 함수마다 이벤트를 붙이지 않아 배선이 한 곳(여기)에 모입니다
업로드 줄 비우기	목업에 있던 예시 진행 줄은 진짜 업로드가 아닙니다. 첫 업로드부터 깨끗하게 시작합니다
태그 실패를 삼킴	태그가 없어도 목록·재생은 되어야 합니다. 실패를 `try` 안에서 삼키고 빈 목록으로 둡니다
401 은 조용히	로그인 안 된 상태는 오류 창이 아니라 로그인 흐름이 맡습니다. 그 외 오류만 `showToast` 로 알립니다
실패 시 예시 줄 유지	`renderSampleList` 는 성공했을 때만 호출됩니다. 서버가 없으면 HTML 예시 줄이 그대로 남아 화면이 비지 않습니다

4. 시험

확인	결과
목록 3줄이 그려지고 업로드 예시 줄이 사라짐	✓
태그 2개가 `sampleState.tags` 에 담김	✓
콘솔 오류 0건	✓

JS 202 — 샘플 목록 그리기 (`renderSampleList` · `sampleRow`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-ui.js`
작업	S1-07 샘플 라이브러리: 열 번째 함수 (FR-224 목록)
작성	Claude (AI), 2026-10-17, 사용자 검토
브랜치	`feature/phase1/s1-07-sample-library-ui`
다음	203 `drawSampleWaveform`

1. 하는 일

받은 샘플 배열로 `#file-list` 를 다시 그립니다. `studioNx.html` 의 예시 줄을 지우고, 한 건마다 `li.sq-file` 한 줄(`sampleRow`)을 만듭니다. 한 줄에는 재생 버튼 · 아이콘 · 이름+태그 · 길이/키 · 파형 · 삭제 버튼이 들어갑니다. 다 그리면 거르기(204)를 한 번 돌립니다.

```
li.sq-file[data-sample-id][data-kind=audio][data-name][data-bars]
├ button.sq-file__play        ▶ / ❚❚
├ span.sq-file__icon          이름·태그로 고른 이모지
├ span.sq-file__title
│  ├ span.sq-file__name       이름
│  └ span.sq-file__tags       span.sq-file__tag (장르는 is-genre)
├ span.sq-file__meta          "0:05 · Am · 128"
├ span.sq-file__wave          <i>(진행) + JS 가 그리는 <canvas>
└ button.sq-file__del         ×
```

2. 코드

```
function renderSampleList(items, total) {
  const listEl = document.getElementById('file-list');
  if (!listEl) return;
  sampleState.items = items;
  sampleState.total = total;
  listEl.textContent = '';
  items.forEach((item) => listEl.appendChild(sampleRow(item)));
  sampleState.usedBytes = items.reduce((sum, item) => sum + (item.file_size ? item.file_size : 0), 0);
  updateStorageBar(sampleState.usedBytes, sampleState.limitBytes);
  applySampleFilters();
}
```

한 줄 — `sampleRow` (요약)

```
function sampleRow(item) {
  const li = document.createElement('li');
  li.className = 'sq-file';
  li.setAttribute('role', 'option');
  li.setAttribute('aria-selected', 'false');
  li.setAttribute('draggable', 'true');
```

```
  li.dataset.kind = 'audio';
  li.dataset.sampleId = String(item.sample_id);
  li.dataset.name = item.name;
  li.dataset.bars = String(sampleBars(item.duration_ms));
  // 재생 · 아이콘 · 이름+태그 · 메타 · 파형 · 삭제 버튼을 차례로 붙임
  drawSampleWaveform(li);   // 203 — 파형은 나중에 채워짐
  return li;
}
```

3. 왜 이렇게

선택	이유
`#file-list` 를 통째로 다시 그림	한 쪽 분량(≤50줄)이라 값싸고, 순서·개수 어긋남이 없습니다
`draggable="true"` · `data-kind=audio` · `data-name` · `data-bars`	트랙에 끌어 놓을 때 `sequencer-arrange.js`(드래그 셀렉터 `.sq-file[draggable="true"][data-kind]`)가 씁니다. 목업 줄과 같은 이름을 유지해야 합니다
`data-bars` = `sampleBars(duration_ms)`	120 BPM·4/4 기준(1마디 2초)으로 계산한 클립 길이입니다
태그는 이름 옆 작은 칩	목록을 좁히는 단서입니다. 장르 태그만 `is-genre` 색으로 구분합니다
다 그린 뒤 `applySampleFilters()`	검색·필터가 이미 켜져 있으면 새 목록에도 바로 적용됩니다
저장 용량을 여기서 계산	목록을 받은 시점이 값을 아는 유일한 곳입니다. `file_size` 합으로 막대를 채웁니다

4. 시험

확인	결과
목록 3건 → `#file-list .sq-file` 3줄	✓
첫 줄 `data-bars` = 0.25 (500ms → 1/4마디)	✓
저장 용량 820 KB / 1 GB, 막대 0.1%	✓

JS 203 — 목록 줄 파형 그리기 (`drawSampleWaveform` · `drawPeaks`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-ui.js`
작업	S1-07 샘플 라이브러리: 열한 번째 함수 (FR-223 파형)
작성	Claude (AI), 2026-10-17, 사용자 검토
브랜치	`feature/phase1/s1-07-sample-library-ui`
다음	204 `applySampleFilters`

1. 하는 일

줄의 `.sq-file__wave` 안에 캔버스를 만들어, 서버가 준 파형(196 `peaks`)을 세로줄로 그립니다. 파형이 아직 없거나 실패하면 아무것도 하지 않아 기존 CSS 세로 무늬가 그대로 남습니다.

```
.sq-file__wave
├ canvas.sq-file__wave__canvas   ← JS 가 삽입·그림 (있으면 .has-wave)
└ i                              ← 재생 진행 채움(205). 캔버스 위에 그려짐
```

2. 코드

```
async function drawSampleWaveform(row) {
  const wave = row.querySelector('.sq-file__wave');
  const id = row.dataset.sampleId;
  if (!wave || !id) return;
  let data = null;
  try { data = await fetchSampleWaveform(id); } catch (e) { return; }
  const peaks = data && data.peaks ? data.peaks : null;
  if (!peaks || !peaks.length) return;
  const canvas = document.createElement('canvas');
  canvas.className = 'sq-file__wave__canvas';
  wave.classList.add('has-wave');
  wave.insertBefore(canvas, wave.firstChild);
  drawPeaks(canvas, peaks, colorOf());
}

function drawPeaks(canvas, peaks, color) {
  const w = canvas.clientWidth, h = canvas.clientHeight;
  if (w < 2 || h < 2) return;
  const dpr = window.devicePixelRatio || 1;
  canvas.width = Math.max(1, Math.round(w * dpr));
  canvas.height = Math.max(1, Math.round(h * dpr));
  const ctx = canvas.getContext('2d');
  ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
  ctx.clearRect(0, 0, w, h);
  const mid = h / 2;
  ctx.fillStyle = color || 'currentColor';
  const step = peaks.length / w;
  for (let x = 0; x < w; x++) {
```

```
    const peak = peaks[Math.min(peaks.length - 1, Math.floor(x * step))];
    const top = mid - (peak ? peak[1] : 0) * mid;
    const bot = mid - (peak ? peak[0] : 0) * mid;
    ctx.fillRect(x, Math.max(0, top), 1, Math.max(1, bot - top));
  }
}
```

CSS (`studio.css`)

```
.sq-file__wave.has-wave { background: none; mask: none; }              /* CSS 무늬 위에 캔버스 */
.sq-file__wave__canvas  { position: absolute; inset: 0; width: 100%; height: 100%; }
.sq-file__wave i        { z-index: 1; }                                /* 진행 채움은 위 */
.sq-file:not(.is-playing) .sq-file__wave i { display: none; }          /* 정지 줄은 채움 숨김 */
```

3. 왜 이렇게

선택	이유
캔버스는 JS 가 삽입	목록이 동적(202)이라 HTML 에 미리 둘 수 없습니다. 파형이 있을 때만 만듭니다
캔버스를 `<i>` 앞에 삽입	DOM 순서상 `<i>`(진행 채움)가 캔버스 위에 그려집니다. `z-index:1` 로 확실히 합니다
정지 줄은 `<i>` 숨김	목업의 `<i style="--p:0%">` 는 재생 전에도 0% 채움을 그립니다. 재생 중이 아니면 숨겨 파형만 보이게 합니다
`has-wave` 로 CSS 무늬 제거	파형 캔버스가 생기면 아래 CSS 세로 무늬(`mask`)가 비쳐 보입니다. 겹치지 않게 지웁니다
dpr 로 캔버스 확대	맥·레티나에서 선이 흐려지지 않게 CSS 폭 × 화면 배율로 잡고, 그리기는 CSS 좌표로 합니다
실패하면 조용히	파형은 곁들이는 정보라, 없어도 줄은 정상입니다

4. 시험

확인	결과
목록 3줄 각각에 캔버스가 생기고 폭·높이 > 0	✓
`.has-wave` 3줄	✓
콘솔 오류 0건	✓

JS 204 — 목록 거르기 (`applySampleFilters` · `applyFilters` 연결)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-ui.js`
작업	S1-07 샘플 라이브러리: 열두 번째 함수 (FR-224 검색·필터)
작성	Claude (AI), 2026-10-17, 사용자 검토
브랜치	`feature/phase1/s1-07-sample-library-ui`
다음	205 `toggleSamplePreview`

1. 하는 일

검색어·필터 칩·고른 태그로 목록 줄을 숨기고 개수를 셉니다. 이름은 `#` 로 시작하면 태그에서, 아니면 이름에서 찾습니다. 0건이면 빈 목록/결과 없음(209)을 보여 줍니다.

```
#file-search  ─┐
필터 칩(All/Drums/…)─┼─▶ applySampleFilters ─▶ 줄 숨김/보임 + sq-files__count
+ Tag(name)  ─┘                                └─▶ toggleEmptyState (0건일 때)
```

2. 코드

```
function applySampleFilters() {
  const listEl = document.getElementById('file-list');
  if (!listEl) return;
  const searchEl = document.getElementById('file-search');
  const q = (searchEl && searchEl.value ? searchEl.value : '').trim().toLowerCase();
  const chip = document.querySelector('.sq-filters [aria-pressed="true"]');
  const chipName = chip && chip.textContent ? chip.textContent.trim() : '';
  const re = FILTERS[chipName] ? FILTERS[chipName] : null;
  const tagName = sampleState.filterTag ? sampleState.filterTag.toLowerCase() : null;
  const byTag = q.charAt(0) === '#';
  let shown = 0;
  listEl.querySelectorAll('.sq-file').forEach((li) => {
    const nameEl = li.querySelector('.sq-file__name');
    const name = nameEl ? nameEl.textContent.toLowerCase() : '';
    const tags = Array.prototype.map.call(li.querySelectorAll('.sq-file__tag'), (t) => t.textContent.toL
owerCase());
    let ok = true;
    if (q) ok = byTag ? tags.some((t) => t.indexOf(q.slice(1)) !== -1) : name.indexOf(q) !== -1;
    if (ok && re && !re.test(name)) ok = false;
    if (ok && tagName && tags.indexOf(tagName) === -1) ok = false;
    li.hidden = !ok;
    if (ok) shown++;
  });
  const count = document.querySelector('.sq-files__count');
  if (count) count.textContent = shown + ' items';
  toggleEmptyState(sampleState.total, shown);
}
```

기존 함수는 이 함수로 넘김 — `applyFilters`

```
function applyFilters() {
  const list = visibleList();
  if (list && list.id === 'file-list') { applySampleFilters(); return; }
  /* … 다른 목록(Presets·Patterns …)은 예전 규칙대로 … */
}
```

3. 왜 이렇게

선택	이유
이미 받은 줄만 거름(클라이언트)	한 쪽(≤50줄)이라 즉시 반응합니다. 서버를 다시 부르지 않아 입력이 끊기지 않습니다
`#` 접두사로 태그 검색	note 75 의 안내("try #Drums")와 맞춥니다. `#` 없으면 이름 검색입니다
칩은 `FILTERS` 정규식 재사용	목업부터 쓰던 규칙(`Drums: /kick
`applyFilters` 위임	왼쪽 목록에는 샘플 외에 Presets·Patterns 등이 있어(카테고리별 생성 목록) 그쪽은 예전 규칙을 유지합니다. `#file-list` 일 때만 새 함수로 보냅니다
숨김은 `hidden` 속성	CSS `.sq-file[hidden]` 이 `display:none` 이라 레이아웃에서 빠집니다

4. 시험

확인	결과
`kick` 검색 → 1줄('Kick 909')만 보임	✓
`zzz` 검색 → 결과 없음 표시, 안내 문구 'zzz'	✓
`Loops` 칩 → 0줄 + 결과 없음 표시	✓
`All` 로 되돌리면 3줄	✓
`+ Tag` 1회 → '#Techno', 'Vocal Chop' 1줄만 보임	✓

JS 205 — 미리듣기 (`toggleSamplePreview` · `stopSamplePreview` · `setSamplePlayGlyph`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-ui.js`
작업	S1-07 샘플 라이브러리: 열세 번째 함수 (FR-224 미리듣기)
작성	Claude (AI), 2026-10-17, 사용자 검토
브랜치	`feature/phase1/s1-07-sample-library-ui`
다음	206 `uploadQueue`

1. 하는 일

줄의 ▶ 를 누르면 그 샘플을 재생하고, 다시 누르면 멈춥니다. 한 번에 하나만 재생하며, 재생 중에는 파형 채움(`--p`)이 현재 위치를 따라갑니다.

```
▶ 클릭 → toggleSamplePreview
  ├ 같은 줄이면 → stopSamplePreview
  ├ fetchSampleFileUrl(id)  ← 195 (blob → objectURL)
  ├ new Audio(url).play()
  └ timeupdate → .sq-file__wave i 의 --p = 현재/전체
    'ended'  → stopSamplePreview
```

2. 코드

```
async function toggleSamplePreview(row) {
  const id = row.dataset.sampleId;
  if (!id) return;
  if (sampleState.playingId === id) { stopSamplePreview(); return; }
  stopSamplePreview();
  let url = null;
  try { url = await fetchSampleFileUrl(id); } catch (e) { showToast('Could not load the sample'); retur
n; }
  const audio = new Audio(url);
  sampleState.audio = audio;
  sampleState.playingId = id;
  row.classList.add('is-playing');
  setSamplePlayGlyph(row, true);
  const fill = row.querySelector('.sq-file__wave i');
  audio.addEventListener('timeupdate', () => {
    if (!audio.duration) return;
    const p = Math.min(100, (audio.currentTime / audio.duration) * 100);
    if (fill) fill.style.setProperty('--p', p.toFixed(1) + '%');
  });
  audio.addEventListener('ended', () => stopSamplePreview());
  const played = audio.play();
  if (played && played.catch) played.catch(() => stopSamplePreview());
}

function stopSamplePreview() {
```

```
  const id = sampleState.playingId;
  if (sampleState.audio) { sampleState.audio.pause(); sampleState.audio = null; }
  sampleState.playingId = null;
  if (id == null) return;
  const row = document.querySelector('#file-list .sq-file[data-sample-id="' + id + '"]');
  if (!row) return;
  row.classList.remove('is-playing');
  setSamplePlayGlyph(row, false);
  const fill = row.querySelector('.sq-file__wave i');
  if (fill) fill.style.setProperty('--p', '0%');
}

function setSamplePlayGlyph(row, playing) {
  const btn = row.querySelector('.sq-file__play');
  if (!btn) return;
  btn.textContent = playing ? '▮▮' : '▶';
  btn.setAttribute('aria-label', (playing ? 'Pause ' : 'Preview ') + (row.dataset.name || 'sample'));
}
```

3. 왜 이렇게

선택	이유
`sampleState.playingId` 하나로 관리	동시에 한 곡만 재생하게 합니다. 다른 줄을 누르면 먼저 멈춥니다
`is-playing` 클래스	CSS 가 이 클래스로 재생 파형 채움(`<i>`)을 보이게 합니다(203)
`--p` 로 진행 표시	목업의 파형 채움과 같은 변수라 따로 그리지 않습니다
`play()` 실패를 `catch`	브라우저 정책·디코딩 실패로 재생이 안 되면 화면만 되돌립니다(오류 창 없음)
끝나면 상태 초기화	'ended' 에서 채움을 0% 로, 글리프를 ▶ 로 되돌립니다

4. 시험

확인	결과
▶ 클릭 시 `/samples/{id}/file` 요청 발생 (1회)	✓
재생 중 `.is-playing` · 글리프 ▮▮ (파일 로드 후)	✓ (설계)
다시 클릭·'ended' 시 ▶ 로 복귀, 채움 0%	✓ (설계)

JS 206 — 업로드 줄 세우기 (`uploadQueue`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-ui.js`
작업	S1-07 샘플 라이브러리: 열네 번째 함수 (FR-226 업로드)
작성	Claude (AI), 2026-10-17, 사용자 검토
브랜치	`feature/phase1/s1-07-sample-library-ui`
다음	207 `renderUploadRow`

1. 하는 일

고르거나 끌어다 놓은 파일들을 하나씩 올립니다. 파일마다 진행 줄(207)을 만들고, 진행률은 `uploadSample`(194)의 콜백으로 채웁니다. 취소(×)를 누르면 그 파일만 멈춥니다. 성공하면 줄을 지우고 새 샘플을 목록 맨 위에 넣고 저장 용량을 더합니다.

```
Upload 버튼 / 드롭 / 빈 목록 버튼 → uploadQueue(files)
  파일마다:
    renderUploadRow → li.sq-upload-row        (207)
    uploadSample(file, { signal, onProgress }) (194)
      progress → setUploadProgress(줄, %)
      성공 → 줄 지움 + prependSampleRow + 저장 용량 갱신
      취소 → 줄 지움
      실패 → 줄에 is-error + 짧은 이유 (uploadErrorMessage)
  줄이 하나도 안 남으면 #upload-list 숨김
```

2. 코드

```
function uploadQueue(fileList) {
  const listEl = document.getElementById('upload-list');
  if (!listEl) return;
  const files = Array.prototype.slice.call(fileList);
  if (!files.length) return;
  listEl.hidden = false;
  files.forEach((file) => {
    const row = renderUploadRow(file);
    const controller = new AbortController();
    const cancel = row.querySelector('.sq-upload-row__cancel');
    if (cancel) cancel.addEventListener('click', () => controller.abort());
    uploadSample(file, {
      signal: controller.signal,
      onProgress: (loaded, total) => setUploadProgress(row, loaded, total),
    }).then((sample) => {
      row.remove();
      if (!listEl.children.length) listEl.hidden = true;
      if (sample) {
        prependSampleRow(sample);
        sampleState.usedBytes += sample.file_size ? sample.file_size : 0;
        sampleState.total += 1;
```

```
        updateStorageBar(sampleState.usedBytes, sampleState.limitBytes);
        applySampleFilters();
      }
      showToast(file.name + ' uploaded');
    }).catch((err) => {
      if (err && err.code === 'ABORTED') row.remove();
      else { row.classList.add('is-error'); row.querySelector('em').textContent = uploadErrorMessage(er
r); }
      if (!listEl.children.length) listEl.hidden = true;
    });
  });
}
```

3. 왜 이렇게

선택	이유
파일마다 `AbortController`	취소(×)는 그 파일 하나만 멈춰야 합니다. 한 번에 여러 파일이 올라가도 서로 영향이 없습니다
`Array.prototype.slice.call`	`FileList` 는 배열이 아니어서 `forEach` 를 못 씁니다(전통 문법 유지)
성공 시 `prependSampleRow`	서버가 돌려준 그 샘플을 목록 맨 위에 넣어, 다시 목록을 받지 않고도 바로 보이게 합니다
실패 줄은 남겨 둠	왜 실패했는지(`Too large` 등)를 보여 줍니다. 취소만 줄을 지웁니다
빈 목록은 숨김	진행 줄이 하나도 없으면 `#upload-list` 를 감춥니다

4. 시험

확인	결과
파일 1개 드롭 → 줄이 1개 생기고 0% 표시	✓
끝나면 줄이 사라지고 `#upload-list` 가 숨겨짐	✓
새 샘플 'New Loop' 이 목록 맨 위에 붙고 개수가 3으로 복귀	✓

JS 207 — 업로드 진행 줄 (`renderUploadRow` · `setUploadProgress` · `uploadErrorMessage`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-ui.js`
작업	S1-07 샘플 라이브러리: 열다섯 번째 함수 (FR-226 진행 표시)
작성	Claude (AI), 2026-10-17, 사용자 검토
브랜치	`feature/phase1/s1-07-sample-library-ui`
다음	208 `updateStorageBar`

1. 하는 일

업로드 중인 파일 하나의 진행 줄(`li.sq-upload-row`)을 만들어 `#upload-list` 에 붙입니다. 진행률을 채우고, 실패하면 짧은 이유를 보여 줍니다.

```
li.sq-upload-row
├ span.sq-upload-row__ic        🎵
├ b                             파일 이름
├ em                            "0%" → "100%" (실패 시 "Too large" 등)
├ button.sq-upload-row__cancel ×
└ span.sq-upload-row__bar > i  --p 로 채움
```

2. 코드

```
function renderUploadRow(file) {
  const listEl = document.getElementById('upload-list');
  const li = document.createElement('li');
  li.className = 'sq-upload-row';

  const icon = document.createElement('span');
  icon.className = 'sq-upload-row__ic';
  icon.setAttribute('aria-hidden', 'true');
  icon.textContent = '🎵';
  li.appendChild(icon);

  const name = document.createElement('b');
  name.textContent = file.name;
  li.appendChild(name);

  const state = document.createElement('em');
  state.textContent = '0%';
  li.appendChild(state);

  const cancel = document.createElement('button');
  cancel.type = 'button';
  cancel.className = 'sq-upload-row__cancel';
```

```
  cancel.setAttribute('aria-label', 'Cancel upload');
  cancel.textContent = '×';
  li.appendChild(cancel);

  const bar = document.createElement('span');
  bar.className = 'sq-upload-row__bar';
  const fill = document.createElement('i');
  fill.style.setProperty('--p', '0%');
  bar.appendChild(fill);
  li.appendChild(bar);

  if (listEl) listEl.appendChild(li);
  return li;
}

function setUploadProgress(row, loaded, total) {
  const p = total ? Math.min(100, Math.round((loaded / total) * 100)) : 0;
  const fill = row.querySelector('.sq-upload-row__bar i');
  if (fill) fill.style.setProperty('--p', p + '%');
  const state = row.querySelector('em');
  if (state) state.textContent = p + '%';
}

function uploadErrorMessage(err) {
  if (!err) return 'Upload failed';
  if (err.code === 'STORAGE_QUOTA_EXCEEDED') return 'No space left';
  if (err.code === 'FILE_TOO_LARGE') return 'Too large';
  if (err.code === 'UNSUPPORTED_FILE_TYPE' || err.code === 'UNSUPPORTED_MEDIA_TYPE') return 'Unsupported
format';
  if (err.code === 'NETWORK_ERROR') return 'Network error';
  return err.message ? err.message : 'Upload failed';
}
```

3. 왜 이렇게

선택	이유
목업과 같은 클래스	`studio.css` 의 `.sq-upload-row` 격자(아이콘 / 이름 / 글자 / 버튼 + 아래 막대)를 그대로 씁니다
진행률은 `--p` 변수	목업의 막대가 `--p` 로 채워집니다. 캔버스 없이 CSS 만으로 진행을 그립니다
짧은 이유로 바꿈	서버 메시지가 길 수 있어, 자주 나오는 코드 몇 개만 짧게 줄입니다. 나머지는 메시지를 그대로 씁니다
`em` 하나로 %·이유	같은 자리에 퍼센트를 보였다가 실패하면 이유를 보여 줍니다(공간 절약)

4. 시험

확인	결과
업로드 줄이 생기고 `em` 0% → (진행) → 사라짐	✓
`is-error` 일 때 `em` 색이 빨감 (CSS `.sq-upload-row.is-error em`)	✓ (설계)

JS 208 — 저장 용량 표시 (`updateStorageBar` · `formatBytes` · `parseBytes` · `readStorageLimit`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-ui.js`
작업	S1-07 샘플 라이브러리: 열여섯 번째 함수 (FR-227 저장 용량)
작성	Claude (AI), 2026-10-17, 사용자 검토
브랜치	`feature/phase1/s1-07-sample-library-ui`
다음	209 `toggleEmptyState`

1. 하는 일

패널 아래 "Project storage" 표시를 실제 값으로 채웁니다. 쓴 양은 목록 샘플의 `file_size` 합, 최대는 화면에 적힌 "… / 1 GB" 에서 읽은 값입니다.

```
#storage
├─ em.sq-storage__val   "820 KB / 1 GB"
└─ .sq-storage__bar i   --p: 0.1%
```

2. 코드

```
function updateStorageBar(usedBytes, limitBytes) {
  const storage = document.getElementById('storage');
  if (!storage) return;
  const label = document.querySelector('#storage .sq-storage__val');
  const bar = document.querySelector('#storage .sq-storage__bar i');
  const limit = limitBytes ? limitBytes : sampleState.limitBytes;
  const pct = limit ? Math.min(100, (usedBytes / limit) * 100) : 0;
  if (label) label.textContent = formatBytes(usedBytes) + ' / ' + formatBytes(limit);
  if (bar) bar.style.setProperty('--p', pct.toFixed(1) + '%');
}

function formatBytes(bytes) {
  if (bytes == null) return '—';
  const mb = bytes / (1024 * 1024);
  if (mb >= 1024) return (mb / 1024).toFixed(1).replace(/\.0$/, '') + ' GB';
  if (mb >= 1) return Math.round(mb) + ' MB';
  return Math.round(bytes / 1024) + ' KB';
}

function parseBytes(text) {
  const m = /([\d.]+)\s*(GB|MB|KB|B)/i.exec(text ? text : '');
  if (!m) return 0;
  const n = parseFloat(m[1]);
  const unit = m[2].toUpperCase();
  if (unit === 'GB') return n * 1024 * 1024 * 1024;
```

```
  if (unit === 'MB') return n * 1024 * 1024;
  if (unit === 'KB') return n * 1024;
  return n;
}

function readStorageLimit() {
  const label = document.querySelector('#storage .sq-storage__val');
  if (!label) return;
  const parts = label.textContent.split('/');
  if (parts.length === 2) {
    const limit = parseBytes(parts[1]);
    if (limit) sampleState.limitBytes = limit;
  }
}
```

3. 왜 이렇게

선택	이유
쓴 양 = `file_size` 합	남은 용량 API(`GET /me/storage`, FR-134)가 아직 없습니다. 한 쪽(≤50)을 다 받으므로 목록 합이 곧 쓴 양입니다. API 가 생기면 그 값으로 바꿉니다
최대는 화면 글자에서 읽음	상수 1 GB 를 코드에 박지 않고, 표기값(`… / 1 GB`)을 기준으로 삼습니다. 표기가 바뀌면 따라갑니다
1024 기준 (KB/MB/GB)	파일 크기 표기 관례(2진)를 따릅니다. `file_size` 도 바이트 단위입니다(177 문서)
100% 로 자름	한도가 작아 넘겨도 막대는 가득찬 모양까지만 보여 줍니다

> 남은 용량 API(FR-134)가 아직 없습니다. 지금 값은 "받아 온 목록의 합"이라 50건을 넘는 프로젝트에서는 실제보다 작게 보입니다. `GET /me/storage` 가 생기면 `updateStorageBar(api.used, api.limit)` 로 바꾸면 됩니다.

4. 시험

확인	결과
목록 3건(합 840000B) → "820 KB / 1 GB", 막대 0.1%	✓
1건 삭제(120000B) → "703 KB / 1 GB"	✓
업로드로 늘어난 용량이 더해짐	✓

JS 209 — 빈 목록 · 결과 없음 (`toggleEmptyState`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-ui.js`
작업	S1-07 샘플 라이브러리: 열일곱 번째 함수 (FR-221·224 빈 상태)
작성	Claude (AI), 2026-10-17, 사용자 검토
브랜치	`feature/phase1/s1-07-sample-library-ui`
다음	210 `startSampleEdit`

1. 하는 일

목록이 0건일 때 알맞은 안내를 보여 줍니다. 전체가 0건이면 "샘플 없음"(업로드 유도), 전체는 있는데 거른 결과가 0건이면 "결과 없음"(검색 지우기)을 띄웁니다.

```
전체 0건          → #files-empty 보임 (Upload files)
전체>0, 보임 0건  → #files-noresults 보임 ("No samples match “<검색어>”")
그 외             → 둘 다 숨김
```

2. 코드

```
function toggleEmptyState(total, shown) {
  const empty = document.getElementById('files-empty');
  const none = document.getElementById('files-noresults');
  if (!empty || !none) return;
  empty.hidden = total > 0;
  const noResults = total > 0 && shown === 0;
  none.hidden = !noResults;
  if (noResults) {
    const query = document.getElementById('noresults-query');
    const searchEl = document.getElementById('file-search');
    if (query) query.textContent = searchEl ? searchEl.value.trim() : '';
  }
}
```

3. 왜 이렇게

선택	이유
두 상태를 구분	빈 패널에 "검색을 지우세요"는 말이 안 됩니다. 상황에 맞는 안내를 보여 줍니다
`hidden` 속성	CSS `.sq-empty[hidden]{display:none}` 로 감춥니다. 목록 영역 자리(008)는 그대로 둡니다
검색어를 안내에 넣음	"No samples match “zzz”" 처럼 무엇을 찾았는지 되짚어 줍니다

4. 시험

확인	결과
3건 정상 → 둘 다 숨김	✓
`zzz` 검색 → 결과 없음 보임, 문구 'zzz'	✓
`Loops` 칩 → 결과 없음 보임	✓

JS 210 — 이름·태그 편집 · 삭제 · 버튼 연결 (`startSampleEdit` · `saveSampleEdit` · `removeSample` · `bindSamplePanelEvents`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-ui.js`
작업	S1-07 샘플 라이브러리: 열여덟 번째 함수 (FR-225 이름·태그·삭제, FR-224 연결)
작성	Claude (AI), 2026-10-17, 사용자 검토
브랜치	`feature/phase1/s1-07-sample-library-ui`
다음	함수 문서 합본 PDF · 개요 ai-note 76 · 커밋

1. 하는 일

줄을 더블 클릭하면 아래에 편집 상자를 열어 이름과 태그를 고칩니다. Save 를 누르면 바뀐 것만 서버로 보내고(197·198) 그 줄만 다시 그립니다. 줄의 × 로 샘플을 지웁니다(199). 이 모든 버튼을 `bindSamplePanelEvents` 가 한 곳에서 연결합니다.

```
#file-list  dblclick  → startSampleEdit(li)
   li.sq-file-edit
   ├ input.sq-file-edit__name
   ├ div.sq-file-edit__tags > button.sq-file-edit__tag[aria-pressed]
   └ button.sq-link(Cancel) / button.sq-upload.sq-file-edit__save(Save)
   Save → saveSampleEdit → renameSample? + setSampleTags? → replaceSampleRow

#file-list  click .sq-file__del → removeSample → confirm → deleteSample → 줄 제거

bindSamplePanelEvents (201 에서 한 번)
  ├ #file-list click/delegate (삭제 · 재생)
  ├ #file-list dblclick (편집)
  ├ #file-search input (150ms 디바운스 → 204)
  ├ .sq-filters click (칩 · + Tag 순환)
  ├ #upload-button · #empty-upload · #upload-input change (→ 206)
  ├ #drop-zone dragover/dragleave/drop (→ 206)
  └ #clear-search click
```

2. 코드

```
function startSampleEdit(row) {
  const id = row.dataset.sampleId;
  const listEl = document.getElementById('file-list');
  if (!id || !listEl) return;
  const existing = listEl.querySelector('.sq-file-edit');
  if (existing) existing.remove();
  const item = findSampleItem(id);
  if (!item) return;
  const box = document.createElement('li');
  box.className = 'sq-file-edit';
```

```
  box.dataset.sampleId = id;
  const nameInput = document.createElement('input');
  nameInput.type = 'text';
  nameInput.className = 'sq-file-edit__name';
  nameInput.maxLength = 200;
  nameInput.value = item.name;
  box.appendChild(nameInput);
  // 태그 버튼: /tags 목록으로 만들고, 붙은 것은 aria-pressed="true"
  const tagWrap = document.createElement('div');
  tagWrap.className = 'sq-file-edit__tags';
  const itemTagIds = (item.tags ? item.tags : []).map((t) => String(t.tag_id));
  sampleState.tags.forEach((t) => {
    const btn = document.createElement('button');
    btn.type = 'button';
    btn.className = 'sq-file-edit__tag';
    btn.dataset.tagId = String(t.tag_id);
    btn.setAttribute('aria-pressed', itemTagIds.indexOf(String(t.tag_id)) !== -1 ? 'true' : 'false');
    btn.textContent = t.name;
    btn.addEventListener('click', () => btn.setAttribute('aria-pressed', btn.getAttribute('aria-presse
d') === 'true' ? 'false' : 'true'));
    tagWrap.appendChild(btn);
  });
  box.appendChild(tagWrap);
  // Cancel / Save
  // … 생략 …
  row.after(box);
  nameInput.focus();
  nameInput.select();
}

async function saveSampleEdit(row, box, item) {
  const name = box.querySelector('.sq-file-edit__name').value.trim();
  const tagIds = Array.prototype.map.call(box.querySelectorAll('.sq-file-edit__tag[aria-pressed="tru
e"]'), (b) => Number(b.dataset.tagId));
  const currentIds = (item.tags ? item.tags : []).map((t) => String(t.tag_id)).sort().join(',');
  const nextIds = tagIds.map((n) => String(n)).sort().join(',');
  let updated = item;
  if (name && name !== item.name) updated = await renameSample(item.sample_id, name);
  if (currentIds !== nextIds) updated = await setSampleTags(item.sample_id, tagIds);
  replaceSampleRow(row, updated ? updated : item);
  box.remove();
  showToast('Sample updated');
}

function removeSample(row) {
  const id = row.dataset.sampleId;
  const item = findSampleItem(id);
  if (!id || !item) return;
  if (!window.confirm('Delete "' + item.name + '"?')) return;
  deleteSample(id).then(() => {
    if (sampleState.playingId === id) stopSamplePreview();
    sampleState.items = sampleState.items.filter((s) => String(s.sample_id) !== String(id));
    sampleState.usedBytes = Math.max(0, sampleState.usedBytes - (item.file_size ? item.file_size : 0));
    sampleState.total = Math.max(0, sampleState.total - 1);
    row.remove();
    updateStorageBar(sampleState.usedBytes, sampleState.limitBytes);
    applySampleFilters();
```

```
  });
}
```

편집 상자 열기 CSS (`studio.css`)

```
.sq-file-edit { display: grid; gap: 0.35rem; padding: 0.45rem 0.65rem; border-bottom: 1px solid var(--sq-line); background: var(--sq-panel-2); }
.sq-file-edit__tag[aria-pressed="true"] { background: color-mix(in srgb, var(--sq-accent) 26%, transparent); border-color: var(--sq-accent); color: var(--c-ink); }
```

3. 왜 이렇게

선택	이유
상자를 목록의 `li` 로	목록이 세로로 늘어서므로, 편집 상자도 줄 사이에 끼워 넣어 위치가 자연스럽습니다
한 번에 하나만	새로 열면 이전 상자를 지웁니다. 열린 상자가 여러 개면 어느 걸 저장하는지 헷갈립니다
바뀐 것만 보냄	이름만 바꿨으면 `PATCH` 만, 태그만 바꿨으면 `PUT` 만 보냅니다. 불필요한 요청이 없습니다
태그는 누름 버튼	태그가 많아도 목록에서 골라 켜고 끌 수 있습니다. 저장 시 켜진 것의 번호를 모읍니다
지우기 전 `confirm`	되돌릴 수 없는 작업이라 한 번 묻습니다
이벤트 위임 (delegate)	줄이 다시 그려져도(202) 위임이라 다시 붙일 필요가 없습니다. 배선이 `bindSamplePanelEvents` 한 곳에 모입니다
검색 150ms 디바운스	타자마다 거르면 느립니다. 잠깐 멈추면 한 번만 돌립니다
드롭은 `Files` 일 때만	트랙 클립을 끌어오는 내부 드래그와 구분합니다

4. 시험

확인	결과
더블 클릭 → 상자 열림(이름 'Kick 909', 태그 버튼 2개, 붙은 것 1개 눌림)	✓
이름 바꾸고 태그 토글 후 Save → `PATCH` + `PUT` 호출, 줄 이름 'Kick 909 v2'	✓
× → 확인 → `DELETE`, 줄 3 → 2, 저장 용량 820 → 703 KB	✓
`+ Tag` 클릭 → '#Techno' 눌림, 'Vocal Chop' 1줄만 보임	✓
검색 입력 → 150ms 뒤 1회 필터	✓
파일 드롭 → `#upload-list` 에 진행 줄	✓
콘솔 오류 0건	✓